

Intel·ligència Artificial

PRÀCTICA 3

Mario VILAR

20392724

David Díez

20392643

12 de desembre de 2023

Professor: Ignasi Cos



UNIVERSITAT DE
BARCELONA

ÍNDEX

1 Introducció

2 Cos del treball

3 Conclusions

Referències

2

2

17

17

$$f(n) = \begin{cases} 0 & \text{if } n = 0 \\ 1 & \text{if } n = 1 \\ f(n-1) + f(n-2) & \text{if } n \geq 2 \end{cases}$$

INTRODUCCIÓ

The goal of this practical is to familiarise ourselves with Q-learning, a basic Reinforcement Learning (RL) algorithm. RL algorithms are a family of powerful methods to solve complex problems in a semi-supervised fashion. In other words, by contrast to neural networks, which learn on the basis of examples, RL occurs by means of a scalar signal (reward). As a general rule, you may consider that any RL implementation must abide by several requirements:

1. First, the problem is defined as a Markov Decision Problem, where the actions of the agent do not in general determine the outcome (they can have a stochastic outcome).
2. Second, the definition of the state is crucial, as it can vastly alter the problem, facilitating its solution or rendering it intractable. Because of this, you should devote some time to think of what the best definition of the state to use for your specific problem. For example, in the case of chess (like the one of our practical), the problem consists of finding a policy that leads to a check mate. As a state, we could use a list containing the positions and types of pieces, or the entire board. The problem may be solved in either case, but the amount of memory and calculations required varies largely in either case.
3. Third, solving the problem is equivalent to finding the right policy to transition across states, starting from an initial state to a final state (which is not necessarily known beforehand).
4. Fourth, since the “right” (optimal) policy is the one leading to maximum reward, the manner in which we assign reward (the so-called reward function) is also crucial to find the desired state.
5. Five, RL algorithms learn on the basis of trial-and-error assessment, quantified in term of reward. In other words, learning occurs when the prediction of expected reward associated to a particular transition differs from the real reward obtained when executing the transition. Conversely, the algorithm is said to converge when the rewards obtained approach expected values.
6. Sixth, the definition of the reward assignment function (the one quantifying the reward associated to each transition) often requires significant introspection, as it may yield radically different solutions to the same problem. For example, if you want to find the shortest path from point A to point B in a grid, you may assign a negative value at each intermediate step, and a large value when reaching B.

COS DEL TREBALL

Observació 2.1 (Observacions inicials, fetes a classe).

- Molts dels escenaris en la pràctica anterior eren completament deterministes. En aquests problemes, un estat s_k està unívocament determinat; això és, per a cada s_k hi haurà un únic estat successor s_{k+1} . No passa el mateix en un escenari estocàstic, ja que per a cada s_k podem tenir diversos successors s_{k+1} .

- Hi ha escenaris que són lleugerament estocàstics. El problema del *drunken sailor*, que ens ocuparà un gruix considerable d'aquesta pràctica, és intrínscament estocàstic.
- Les polítiques no són Q -learning. Q , en definitiva, és la funció de valor i la política, i triarem aquella que tingui un valor Q més gran. Al principi no coneixem els valors Q i cal calcular-los.
- Per veure si el nostre algorisme està aprenent hauríem de monitoritzar la δ i veure que aquesta es va acostant a zero uniformement. Problemes completament estocàstics com el *drunken sailor* tenen unes condicions d'aprenentatge especialment favorables (si normalment $0 < \epsilon < 1$, per al *drunken sailor* podríem tenir $\epsilon = 1$ i el nostre programa seguiria aprenent).

Exercici 2.2.

1. *Implement the Q -learning algorithm to find the optimal path considering a reward of -1 everywhere except for the goal, with reward 100.*
 1. *Print the first, two intermediate and the final Q -table. What sequence of actions do you obtain?*
 2. *After trying for a bit, what is your parameter choice for alpha, gamma and epsilon? Why?*
 3. *How do you judge convergence of the algorithm? How long does it take to converge?*
2. *Try implementing the more accurate reward.*
 1. *Answer the questions of the previous section for this case.*
 2. *What is the effect of the new reward function on performance?*
 3. *How does this relate to the search algorithms studied in P1? Could you apply one of those in this case?*
3. *The main novelty in RL algorithms with respect to the search algorithms in P1 is that they can be applied in stochastic environments, where the agent doesn't fully determine the outcome of its actions.*
 1. *Drunken sailor. Your agent is now a drunken sailor trying to get to bed after a good share of whiskey and shanties: their legs don't seem to obey them all the time. Introduce stochasticity (= randomness) by enforcing that only 99% of the steps intended by the sailor are actually taken, the rest leading randomly in any other possible direction.*
 2. *Use at least one of the two rewards proposed:*
 1. *What is your parameter choice? Why?*
 2. *Assuming the sailor is in a state that allows learning: how many drunken nights are necessary for them to master the perilous path to bed? Compare to the previous, deterministic scenario.*
 3. *What is the optimal path found? If we watched the sailor try to follow it, would they always follow the same path?*
 4. *Could we apply one of the algorithms in P1 here? Why? Hint: think of the notions of deterministic vs random and of path vs policy.*

Demostració.

I. Pel que fa al primer apartat.

I. La sortida del nostre programa és la següent:

```

1  ITERACIÓN: 1
2  -----
3  |      00.00      |      00.00      |      00.00      |      00.00      |
4  |00.00      00.00|00.00      00.00|00.00      00.00|00.00      00.00|
5  |      00.00      |      00.00      |      00.00      |      00.00      |
6  -----
7  |      00.00      |      00.00      |      00.00      |      00.00      |
8  |00.00      00.00|00.00      00.00|00.00      00.00|00.00      00.00|
9  |      00.00      |      00.00      |      00.00      |      00.00      |
10 -----
11 |      00.00      |      00.00      |      00.00      |      00.00      |
12 |-0.90      00.00|00.00      00.00|00.00      00.00|00.00      00.00|
13 |      00.00      |      00.00      |      00.00      |      00.00      |
14 -----
15
16 ITERACIÓN: 368263
17 -----
18 |      70.19      |      79.10      |      89.00      |      00.00      |
19 |70.19      79.10|70.19      89.00|79.10      100.0|00.00      00.00|
20 |      62.17      |      00.00      |      79.10      |      00.00      |
21 -----
22 |      70.19      |      00.00      |      89.00      |      100.0      |
23 |62.17      00.00|00.00      00.00|00.00      89.00|79.10      89.00|
24 |      54.95      |      00.00      |      70.19      |      79.10      |
25 -----
26 |      62.17      |      00.00      |      79.10      |      89.00      |
27 |54.95      62.17|54.95      70.19|62.17      79.10|70.19      79.10|
28 |      54.95      |      62.17      |      70.19      |      79.10      |
29 -----
30
31 ITERACIÓN: 524857
32 -----
33 |      70.19      |      79.10      |      89.00      |      00.00      |
34 |70.19      79.10|70.19      89.00|79.10      100.0|00.00      00.00|
35 |      62.17      |      00.00      |      79.10      |      00.00      |
36 -----
37 |      70.19      |      00.00      |      89.00      |      100.0      |
38 |62.17      00.00|00.00      00.00|00.00      89.00|79.10      89.00|
39 |      54.95      |      00.00      |      70.19      |      79.10      |
40 -----
41 |      62.17      |      00.00      |      79.10      |      89.00      |
42 |54.95      62.17|54.95      70.19|62.17      79.10|70.19      79.10|
43 |      54.95      |      62.17      |      70.19      |      79.10      |

```

```

44 -----
45
46 RESULTADO FINAL:
47 -----
48 |      70.19      |      79.10      |      89.00      |      00.00      |
49 |70.19      79.10|70.19      89.00|79.10      100.0|00.00      00.00|
50 |      62.17      |      00.00      |      79.10      |      00.00      |
51 -----
52 |      70.19      |      00.00      |      89.00      |      100.0      |
53 |62.17      00.00|00.00      00.00|00.00      89.00|79.10      89.00|
54 |      54.95      |      00.00      |      70.19      |      79.10      |
55 -----
56 |      62.17      |      00.00      |      79.10      |      89.00      |
57 |54.95      62.17|54.95      70.19|62.17      79.10|70.19      79.10|
58 |      54.95      |      62.17      |      70.19      |      79.10      |
59 -----

```

Vist això, la seqüència d'accions obtinguda és ambivalent, ja que des de la casella de sortida en molts casos tenim dos valors de la Q -table que són exactament iguals, si triem U (*up*) o R (*right*): $[U, U, R, R, R]$ o bé $[R, R, R, U, U]$ o bé $[R, R, U, U, R]$ o bé $[R, R, U, R, U]$. Té sentit, ja que fent els prints adients veiem que la que retorna el nostre programa és $[R, R, R, U, U]$ (o, equivalentment, $[(2, 1), (2, 2), (2, 3), (1, 3), (0, 3)]$). Com les recompenses són les mateixes excepte la última la solució és única llevat de determinades permutacions.

- Entendrem que la tria de paràmetres és amb l'objectiu de fer convergir l'algoritme de manera més ràpida, així que ajuntarem aquests dos apartats. La convergència de l'algorisme es dona, en termes de la taula de valors Q , a partir de quin valor n_o la successió de taules $(Q_n)_n$ convergeix puntualment a una taula Q . Com més baix sigui aquest n_o , millor. Això ho farem mitjançant el paràmetre δ .

Observació 2.3. Per saber què és aquesta δ , primer hem de definir $\bar{r}_i$, que es defineix com

$$\bar{r}_i = Q(s_k, a_k) - \max_a Q(s_{k+1}, a)$$

i representa el valor de la recompensa teòrica obtinguda en fer un moviment. Tenint això definit, definim $\delta = r_i - \bar{r}_i$, és a dir, la diferència entre la recompensa real i la teòrica. Per tant, el nostre algoritme convergeix quan aquesta diferència, és a dir, δ , tendeix a zero.

Amb el que acabem de dir com a premisa, minimitzar la δ , la tria que hem fet per a α , γ i ε és: $(\alpha, \gamma, \varepsilon) = (0.9, 0.9, 0.8)$, simplement fent prova i error i *aplicant les definicions de cadascun dels paràmetres amb certa lògica*.

- Per exemple, α modul·la la quantitat d'informació anterior que usem (com més alt més informació anterior utilitza) i ens interessa utilitzar-ne molta.
- Ens interessa recórrer tota la taula de valors Q al principi, per anar veient quin camí és millor poc a poc, de manera que $\varepsilon \neq 1$. Ha de ser, també, proper a 1, ja que quan tenim la informació suficient ens interessa utilitzar-la i no deixar les decisions a mans de l'atzar.
- γ es raona de manera similar. Com diria la Maite, *más vale pájaro en mano que ciento volando*.

Més que el temps que triga en convergir, hem mirat la iteració a partir de la qual podem dir que convergeix. Hem fixat la condició de convergència en $\frac{1}{N}$, on N és el nombre màxim d'iteracions que hem definit, normalment 1000000. *La iteració de convergència ha sigut: 493.*

2. Pel que fa al segon apartat, el que hem fet nosaltres no és definir una funció de recompensa, sinó una *matriu* de recompensa, de manera que ens permet estalviar-nos l'ús de diccionaris, tuples i moltes línies de codi *spaghetti*. Al cap i a la fi, qualsevol aplicació lineal es pot representar de manera natural com una matriu. Esperem que no se'ns penalitzi si no és el que es buscava.

1. Responent les preguntes de la secció anterior, anirem molt més ràpid així que recomanem consultar-la un altre cop.

1. La sortida del nostre programa és la següent:

```

1  ITERACIÓN: 1
2  -----
3  |      00.00      |      00.00      |      00.00      |      00.00      |
4  |00.00      00.00|00.00      00.00|00.00      00.00|00.00      00.00|
5  |      00.00      |      00.00      |      00.00      |      00.00      |
6  -----
7  |      00.00      |      00.00      |      00.00      |      00.00      |
8  |00.00      00.00|00.00      00.00|00.00      00.00|00.00      00.00|
9  |      00.00      |      00.00      |      00.00      |      00.00      |
10 -----
11 |      -3.60      |      00.00      |      00.00      |      00.00      |
12 |00.00      00.00|00.00      00.00|00.00      00.00|00.00      00.00|
13 |      00.00      |      00.00      |      00.00      |      00.00      |
14 -----
15
16 ITERACIÓN: 240160
17 -----
18 |      67.29      |      78.10      |      89.00      |      00.00      |
19 |67.29      78.10|67.29      89.00|78.10      100.0|00.00      00.00|
20 |      56.56      |      00.00      |      78.10      |      00.00      |
21 -----
22 |      67.29      |      00.00      |      89.00      |      100.0      |
23 |56.56      00.00|00.00      00.00|00.00      89.00|78.10      89.00|
24 |      45.90      |      00.00      |      67.29      |      78.10      |

```

```

25 -----
26 |      56.56      |      00.00      |      78.10      |      89.00      |
27 |45.90      56.56|45.90      67.29|56.56      78.10|67.29      78.10|
28 |      45.90      |      56.56      |      67.29      |      78.10      |
29 -----
30
31 ITERACIÓ: 799082
32 -----
33 |      67.29      |      78.10      |      89.00      |      00.00      |
34 |67.29      78.10|67.29      89.00|78.10      100.0|00.00      00.00|
35 |      56.56      |      00.00      |      78.10      |      00.00      |
36 -----
37 |      67.29      |      00.00      |      89.00      |      100.0      |
38 |56.56      00.00|00.00      00.00|00.00      89.00|78.10      89.00|
39 |      45.90      |      00.00      |      67.29      |      78.10      |
40 -----
41 |      56.56      |      00.00      |      78.10      |      89.00      |
42 |45.90      56.56|45.90      67.29|56.56      78.10|67.29      78.10|
43 |      45.90      |      56.56      |      67.29      |      78.10      |
44 -----
45
46 RESULTADO FINAL:
47 -----
48 |      67.29      |      78.10      |      89.00      |      00.00      |
49 |67.29      78.10|67.29      89.00|78.10      100.0|00.00      00.00|
50 |      56.56      |      00.00      |      78.10      |      00.00      |
51 -----
52 |      67.29      |      00.00      |      89.00      |      100.0      |
53 |56.56      00.00|00.00      00.00|00.00      89.00|78.10      89.00|
54 |      45.90      |      00.00      |      67.29      |      78.10      |
55 -----
56 |      56.56      |      00.00      |      78.10      |      89.00      |
57 |45.90      56.56|45.90      67.29|56.56      78.10|67.29      78.10|
58 |      45.90      |      56.56      |      67.29      |      78.10      |
59 -----
60 Ha tardado 5.72 segundos

```

Vist això, la seqüència d'accions obtinguda és ambivalent un altre cop. Tenim: $[U, U, R, R, R]$ o bé $[R, R, R, U, U]$ o bé $[R, R, U, U, R]$ o bé $[R, R, U, R, U]$, amb l'algorisme ementent la sortida:

$$[(2, 1), (2, 2), (2, 3), (1, 3), (0, 3)].$$

2. La tria que hem fet per a α, γ i ε és: $(\alpha, \gamma, \varepsilon) = (0.9, 0.9, 0.8)$, aplicant els raonaments anteriors. *La iteració de convergència ha sigut: 372.*
2. La reward function ve definida pels canvis que hem fet en la taula de recompenses.

1. En la primera taula de recompenses trigava uns $\frac{5.56+5.59+5.62+5.66+5.47}{5} = 5.58$ segons. En la mateixa taula convergeix en la iteració $\frac{415+350+362+312+252}{5} = 339$.
2. En la segona taula de recompenses trigava uns $\frac{5.82+5.74+5.73+5.58+5.24}{5} = 5.62$ segons. En la mateixa taula convergeix en la iteració $\frac{113+295+304+265+291}{5} = 254$.
3. El *Q*-learning utilitza una *Q*-table per enregistrar i actualitzar els valors que representen l'esperança de la recompensa futura en funció de les accions preses. Això permet aprendre una política òptima sense conèixer explícitament el model de l'entorn. En canvi, els algorismes com A^* es basen en l'exploració determinista, sense requerir un coneixement previ sobre l'entorn o els moviments possibles per aprendre. El *link*, evidentment, és el fet d'utilitzar la mateixa heurística. Aquesta funció de recompensa basada en heurística guia el procés d'aprenentatge cap a la política òptima, de la mateixa manera que la funció heurística guia A^* cap a l'objectiu de manera eficient. ■
3. Pel que fa al tercer apartat.
 1. Hem definit un paràmetre *obey*, que el que fa és regular les vegades que el sailor farà cas a l'acció que és triada pel nostre algorisme. Com més baixa sigui la taxa d'obediència, més estocàstic serà l'entorn. Està tot al codi, recomanem consultar-lo.
 2. Utilitzant la taula més precisa de recompenses (la que té diferents valors i no sol -1):
 1. Haurem de baixar els paràmetres γ i α en la mesura que forcem un entorn estocàstic; **amb un *obey* baix**, ni les accions anteriors seran especialment rellevants (α baixa), ni tampoc ho seran els estats llunyans (γ relativament baixa), ja que el criteri de tria és virtualment aleatori. Els paràmetres que hem posat són: $(\alpha, \gamma) = (0.2, 0.3)$. Amb un *obey* alt, 0.99 per exemple, la γ i l' α s'hauran d'apropar força al cas determinístic per raons evidents; no acabarà de ser del tot òptim, però ens serveix com a referència. A mesura que l'*obey* baixa, hauren d'anar adaptant els paràmetres.
 2. Que el sailor estigui en un estat que permet aprendre és quan *obey* és estrictament més gran que 0; és a dir, en algun moment s'anima a seguir la recomanació de l'algorisme en quant a l'acció que cal triar. I també, $\alpha > 0$, ja que α té en compte si l'algorisme aprèn o no. Amb *drunken nights* entenem que es refereix a les iteracions, i ens demana, de fet, el nombre d'iteracions pel qual l'algorisme aprèn el suficient com per convergir. De fet, *no només a quina iteració convergeix, sinó quantes vegades arriba a l'estat de destí i completa una ruta*. Amb un *obey* al 0.99, i $(\alpha, \gamma) = (0.2, 0.3)$, obtenim:

```

1  ITERACIÓN: 1
2  -----
3  |    00.00    |    00.00    |    00.00    |    00.00    |
4  |00.00    00.00|00.00    00.00|00.00    00.00|00.00    00.00|

```

```

5 |      00.00      |      00.00      |      00.00      |      00.00      |
6 -----
7 |      00.00      |      00.00      |      00.00      |      00.00      |
8 |00.00      00.00|00.00      00.00|00.00      00.00|00.00      00.00|
9 |      00.00      |      00.00      |      00.00      |      00.00      |
10 -----
11 |      00.00      |      00.00      |      00.00      |      00.00      |
12 |00.00      00.00|00.00      00.00|00.00      00.00|00.00      00.00|
13 |      -1.00      |      00.00      |      00.00      |      00.00      |
14 -----
15 El algoritmo converge en la iteración 3088
16
17 ITERACIÓN: 15011
18 -----
19 |      -0.99      |      06.70      |      29.00      |      00.00      |
20 |-0.99      06.70|-0.99      29.00|06.70      100.0|00.00      00.00|
21 |      -4.00      |      00.00      |      06.70      |      00.00      |
22 -----
23 |      -0.99      |      00.00      |      29.00      |      100.0      |
24 |-4.00      00.00|00.00      00.00|00.00      29.00|06.70      29.00|
25 |      -6.20      |      00.00      |      -0.99      |      06.70      |
26 -----
27 |      -4.00      |      00.00      |      06.70      |      29.00      |
28 |-6.20      -4.00|-6.20      -0.99|-4.00      06.70|-0.99      06.70|
29 |      -6.20      |      -4.00      |      -0.99      |      06.70      |
30 -----
31
32 ITERACIÓN: 587112
33 -----
34 |      -0.99      |      06.70      |      29.00      |      00.00      |
35 |-0.99      06.70|-0.99      29.00|06.70      100.0|00.00      00.00|
36 |      -4.00      |      00.00      |      06.70      |      00.00      |
37 -----
38 |      -0.99      |      00.00      |      29.00      |      100.0      |
39 |-4.00      00.00|00.00      00.00|00.00      29.00|06.70      29.00|
40 |      -6.20      |      00.00      |      -0.99      |      06.70      |
41 -----
42 |      -4.00      |      00.00      |      06.70      |      29.00      |
43 |-6.20      -4.00|-6.20      -0.99|-4.00      06.70|-0.99      06.70|
44 |      -6.20      |      -4.00      |      -0.99      |      06.70      |
45 -----
46
47 RESULTADO FINAL:
48 -----
49 |      -0.99      |      06.70      |      29.00      |      00.00      |
50 |-0.99      06.70|-0.99      29.00|06.70      100.0|00.00      00.00|
51 |      -4.00      |      00.00      |      06.70      |      00.00      |

```

```

52 -----
53 |      -0.99      |      00.00      |      29.00      |      100.0      |
54 |-4.00      00.00|00.00      00.00|00.00      29.00|06.70      29.00|
55 |      -6.20      |      00.00      |      -0.99      |      06.70      |
56 -----
57 |      -4.00      |      00.00      |      06.70      |      29.00      |
58 |-6.20      -4.00|-6.20      -0.99|-4.00      06.70|-0.99      06.70|
59 |      -6.20      |      -4.00      |      -0.99      |      06.70      |
60 -----
61 Ha tardado 4.46 segundos

```

L'algorisme convergeix en la iteració 3088 (les *drunken nights* necessàries que demana) i triga 4,46s. Comparant-ho amb el determinista (obey=1), primer veiem una Q -table radicalment diferent, amb valors negatius i tot, però *el camí òptim segueix estant ben definit*. El determinista, recordem, triga (5,58, 5,62) segons i triga (339, 254) iteracions en convergir, en funció de la taula de recompenses que seguim.

Notem, en particular, que el ràtio de convergència entre l'estocàstic i el determinista és de $\frac{3088}{339} = 9,1$; és a dir, l'estocàstic triga una mitjana de 9,1 vegades més iteracions en convergir que el determinista.

3. Se'ns està preguntant si el component aleatori és prou fort com per alterar el camí òptim que trobem amb el nostre algorisme.

[(2, 1), (2, 0), (2, 1), (2, 1), (2, 0), (2, 1), (2, 1), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)]

[(2, 1), (2, 0), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)]

[(2, 1), (2, 0), (2, 1), (2, 0), (2, 1), (2, 0), (2, 1), (2, 0), (2, 1), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)]

[(2, 1), (2, 0), (2, 1), (2, 1), (2, 0), (2, 1), (2, 1), (2, 2), (2, 3), (1, 3), (0, 3)]

Com veiem, el path que farà és òptim, però hi ha molts estats que es repeteixen a causa de l'entorn, que és estocàstic. Per tant, una resposta més concreta a la pregunta és que no, en efecte, *no se segueix sempre el mateix camí*, ja que les repeticions d'estats que es van produint no estan determinades *determinísticament* (valgui la redundància); és a dir, existeix camí òptim però pel fet que no hem limitat els estats repetits i a causa l'aleatorietat introduïda en la presa de decisions, aquest camí no és únic.

4. Tenint en compte aquesta naturalesa estocàstica inherent al problema del *drunken sailor*, aplicar A^* directament potser no sigui adequat. A^* es basa en una cerca de camins en un entorn determinista, mentre que el problema *drunken sailor* implica més un escenari d'aprenentatge de polítiques estocàstiques, on l'objectiu és determinar la millor política a seguir en un entorn probabilístic.

En altres paraules, en lloc de trobar un camí definit, en el *drunken sailor* necessita desenvolupar una política que maximitzi el retorn esperat o les recompenses al llarg del temps, tenint en compte la naturalesa estocàstica dels moviments. Això és, per definició, el *Q-learning*.

Observació 2.4. A partir d'aquí canviem radicalment la implementació de la *Q-table*. No és viable tenir tanta informació en una matriu, i ho passarem a diccionari, de manera que podrem emmagatzemar valor, millor moviment i moviments possibles d'una manera més natural. Si no es diu el contrari, N el nombre d'iteracions a partir del qual creiem que convergeix, $N = 5$.

Exercici 2.5. *Now, let's move back to the chess scenario, namely the first board configuration of P1. Remember that we have the black king, the white king and one white rook, and that only whites move. Remember to provide the first, two intermediate and the final Q-table in every case.*

1. *Adapt your Q-learning implementation to find the optimal path to a check mate considering a reward of -1 everywhere except for the goal (check mate for the whites), with reward 100.*
 1. *What sequence of actions do you obtain?*
 2. *After trying for a bit, what is your parameter choice for α , γ and ϵ ? Why?*
 3. *How do you judge convergence of the algorithm? How long does it take to converge?*
2. *Try now with a more sensible reward function adapted from the heuristic used for the A^* search:*
 1. *Answer the questions of the previous section for this case.*
 2. *What is the effect of the new reward function on performance?*
3. *Drunken sailor. On their way to bed, our drunken sailor sees a chessboard on a table, coincidentally configured as in the previous section. They have seen the captain play with the first mate and want to give it a try, but only have a rudimentary knowledge of the rules (they know how each piece moves and what is a check mate, but not that blacks move as well).*
 1. *Introduce stochasticity (= randomness) by enforcing that only a given percentage of the moves intended by the sailor are actually taken, the rest taken randomly from all other possibilities.*
 2. *Use any reward you prefer:*
 1. *What is your parameter choice? Why?*
 2. *Assuming our obsessive sailor is in a state that allows learning: how many games do they have to play before they are satisfied that they have found the best strategy and can go to bed? Compare to the previous, deterministic scenario.*
 3. *What is the optimal path found? If we watched the sailor try to follow it, would they always follow the same path?*
 3. *Compare the application of Q-learning in this chess scenario with that of the grid of exercise 1. How do the two scenarios differ? How does that translate into your results?*

4. Compare the use of *Q-learning* with that of search algorithms of P_1 for the chess scenario seen here, both in the deterministic and stochastic case.
5. (Voluntary). Use your *Q-learning* implementation of the drunken sailor on the second board configuration of P_1 , and try to find the best parameter combination for quick convergence. How robust was your previous parameter choice? Discuss your results.

Demostració.

1. Pel que fa al primer apartat.

1. La seqüència d'accions obtinguda és:

$$[[[7, 0, 2], [7, 4, 6]], [[0, 0, 2], [7, 4, 6]], [[6, 3, 6], [0, 0, 2]], [[5, 3, 6], [0, 0, 2]], \\ [[4, 4, 6], [0, 0, 2]], [[3, 3, 6], [0, 0, 2]], [[2, 4, 6], [0, 0, 2]]]$$

2. Com a la pregunta anterior, com més alt α més informació anterior utilitza i ens interessa utilitzar-ne molta; $\varepsilon \neq 1$ i proper a 0, ja que quan tenim la informació suficient ens interessa utilitzar-la i no deixar les decisions a mans de l'atzar. γ es raona de manera similar. La nostra tria ha sigut $(\alpha, \gamma, \varepsilon) = (0.9, 0.9, 0.3)$.
3. La convergència de l'algorisme es fa mitjançant un paràmetre, `prod` en el nostre codi, que es va acumulant i el nombre de moviments de la iteració. És a dir, una mitjana de l'error que es va produint en tots els moviments. Si aquest valor es troba en un entorn que fixem nosaltres, en el nostre cas, de 0.000001, aleshores podem dir que aquella solució en concret ha convergit. Haurem de veure que en iteracions posteriors el programa hagi après el suficient com perquè segueixi convergint. Hem suposat que amb 5 iteracions en teníem suficient per determinar la convergència.

2. Pel que fa al segon apartat.

1. Tornem a respondre les preguntes de l'apartat anterior.

1. La seqüència d'accions obtinguda és:

$$[[[7, 0, 2], [7, 4, 6]], [[6, 3, 6], [7, 0, 2]], [[5, 2, 6], [7, 0, 2]], \\ [[0, 0, 2], [5, 2, 6]], [[4, 2, 6], [0, 0, 2]], [[3, 3, 6], [0, 0, 2]], [[2, 4, 6], [0, 0, 2]]]$$

En particular, la sortida ha estat la següent:

```
1 mariovilar@Marios-MacBook-Pro p3 % python3 aichess.py
2 starting AI chess...
3 Iteration 375 with error 0.0 and total movements 5
4 Iteration 530 with error 0.0 and total movements 5
5 Iteration 649 with error 0.0 and total movements 5
6 Iteration 804 with error 0.0 and total movements 5
7 Iteration 1078 with error 0.0 and total movements 5
```

```

8  Iteration 1079 with error 0.0 and total movements 5
9  *****
10 |   |   |   |   | K |   |   |   |
11 |   |   |   |   |   |   |   |   |
12 |   |   |   |   |   |   |   |   |
13 |   |   |   |   |   |   |   |   |
14 |   |   |   |   |   |   |   |   |
15 |   |   |   |   |   |   |   |   |
16 |   |   |   |   |   |   |   |   |
17 | R |   |   |   | K |   |   |   |
18 *****
19 K moved.
20 ->piece initial state  [7, 4, 6]
21 ->piece to state  [6, 3, 6]
22 *****
23 |   |   |   |   | K |   |   |   |
24 |   |   |   |   |   |   |   |   |
25 |   |   |   |   |   |   |   |   |
26 |   |   |   |   |   |   |   |   |
27 |   |   |   |   |   |   |   |   |
28 |   |   |   |   |   |   |   |   |
29 |   |   |   | K |   |   |   |   |
30 | R |   |   |   |   |   |   |   |
31 *****
32 K moved.
33 ->piece initial state  [6, 3, 6]
34 ->piece to state  [5, 2, 6]
35 *****
36 |   |   |   |   | K |   |   |   |
37 |   |   |   |   |   |   |   |   |
38 |   |   |   |   |   |   |   |   |
39 |   |   |   |   |   |   |   |   |
40 |   |   |   |   |   |   |   |   |
41 |   |   | K |   |   |   |   |   |
42 |   |   |   |   |   |   |   |   |
43 | R |   |   |   |   |   |   |   |
44 *****
45 R moved.
46 ->piece initial state  [7, 0, 2]
47 ->piece to state  [0, 0, 2]
48 *****
49 | R |   |   |   | K |   |   |   |
50 |   |   |   |   |   |   |   |   |
51 |   |   |   |   |   |   |   |   |
52 |   |   |   |   |   |   |   |   |
53 |   |   |   |   |   |   |   |   |
54 |   |   | K |   |   |   |   |   |

```

```

55 | | | | | | | | |
56 | | | | | | | | |
57 *****
58 K moved.
59 ->piece initial state [5, 2, 6]
60 ->piece to state [4, 2, 6]
61 *****
62 | R | | | | K | | | |
63 | | | | | | | | |
64 | | | | | | | | |
65 | | | | | | | | |
66 | | | K | | | | |
67 | | | | | | | | |
68 | | | | | | | | |
69 | | | | | | | | |
70 *****
71 K moved.
72 ->piece initial state [4, 2, 6]
73 ->piece to state [3, 3, 6]
74 *****
75 | R | | | | K | | | |
76 | | | | | | | | |
77 | | | | | | | | |
78 | | | | K | | | |
79 | | | | | | | | |
80 | | | | | | | | |
81 | | | | | | | | |
82 | | | | | | | | |
83 *****
84 K moved.
85 ->piece initial state [3, 3, 6]
86 ->piece to state [2, 4, 6]
87 *****
88 | R | | | | K | | | |
89 | | | | | | | | |
90 | | | | | K | | | |
91 | | | | | | | | |
92 | | | | | | | | |
93 | | | | | | | | |
94 | | | | | | | | |
95 | | | | | | | | |
96 *****
97 Path sequence: [[[7, 0, 2], [7, 4, 6]], [[6, 3, 6], [7, 0, 2]], [[5, 2, 6],
  ↳ [7, 0, 2]], [[0, 0, 2], [5, 2, 6]], [[4, 2, 6], [0, 0, 2]], [[3, 3, 6],
  ↳ [0, 0, 2]], [[2, 4, 6], [0, 0, 2]]]
98 --- It took 40.41 seconds ---

```

Observació 2.6. Hem tret les Q -tables de la sortida perquè no és rellevant pel que volem mostrar. De fet, les posem al codi perquè ens les demana l'enunciat, ja que si no, no ho hauríem fet.

2. Com més alt α més informació anterior utilitza i ens interessa utilitzar-ne molta; $\varepsilon \neq 1$ i proper a 0, ja que quan tenim la informació suficient ens interessa utilitzar-la i no deixar les decisions a mans de l'atzar. γ es raona de manera similar. La nostra tria ha sigut $(\alpha, \gamma, \varepsilon) = (0.9, 0.9, 0.3)$.
3. La convergència de l'algorisme depèn del mateix càlcul, que està fet de manera idèntica a l'apartat anterior. Aleshores, si no es té clar el raonament recomanem revisar-lo un altre cop. Notem, però, que ara els *rewards* canvien i, per tant, si miréssim la taula de Q -values els valors serien diferents.
2. L'efecte de la funció de recompensa en termes de *performance* ha estat significatiu, però tampoc determinant. Això és perquè l'heurística ens dona molta més informació del nostre voltant. Se segueix aprenent; per tant, triga, però ho fa més ràpid. En certa manera, *atrau* als moviments a l'estat que estem buscant. La diferència temporal ha estat d'uns 6 segons, aproximadament.
3. Pel que fa al tercer apartat.
 1. Consultar el codi de la pràctica, on s'ha implementat el que s'ha demanat. Hem de notar la similitud amb el primer exercici i el paràmetre *obey* que comentàvem, que es refereix a la taxa d'accions que obeeirà el *drunken sailor*.
 2. Utilitzant l'heurística com a funció de recompensa:
 1. Haurem de baixar els paràmetres γ i α en la mesura que forcem un entorn estocàstic; **amb un *obey* baix**, ni les accions anteriors seran especialment rellevants (α baixa), ni tampoc ho seran els estats llunyans (γ relativament baixa), ja que el criteri de tria és virtualment aleatori. Els paràmetres que hem posat són: $(\alpha, \gamma) = (0.2, 0.3)$. Amb un *obey* alt, 0.99 per exemple, la γ i l' α s'hauran d'apropar força al cas determinístic per raons evidents; no acabarà de ser del tot òptim, però ens serveix com a referència. A mesura que l'*obey* baixa, hauren d'anar adaptant els paràmetres.
 2. Que el sailor estigui en un estat que permet aprendre és quan *obey* és estrictament més gran que 0; és a dir, en algun moment s'anima a seguir la recomanació de l'algorisme en quant a l'acció que cal triar. I també, $\alpha > 0$, ja que α té en compte si l'algorisme aprèn o no. Ens demana, de fet, el nombre d'iteracions pel qual l'algorisme aprèn el suficient com per convergir. De fet, *no només a quina iteració convergeix, sinó quantes vegades arriba a l'estat de destí i completa una partida amb la millor estratègia*.

1. Amb un obey al 0,99, i $(\alpha, \gamma) = (0.9, 0.9)$ (i $\varepsilon \approx 0.3$), obtenim el camí:

[[[7, 0, 2], [7, 4, 6]], [[0, 0, 2], [7, 4, 6]], [[6, 3, 6], [0, 0, 2]],
[[5, 4, 6], [0, 0, 2]], [[4, 3, 6], [0, 0, 2]], [[3, 3, 6], [0, 0, 2]], [[2, 4, 6], [0, 0, 2]]]

2. Amb un obey al 0,5, posem $(\alpha, \gamma) = (0.2, 0.3)$ (i $\varepsilon \approx 0.5$). Triga molt, així que no posem sortida. Però és totalment normal ja que és molt estocàstic i, naturalment, es triga a arribar a una solució.

3. Se'ns està preguntant si el component aleatori és prou fort com per alterar el camí òptim que trobem amb el nostre algorisme. En efecte, *no se segueix sempre el mateix camí*, existeix camí òptim però a causa l'aleatorietat introduïda en la presa de decisions, aquest camí no és únic i, potser, no és el que ha triat el programa.

1. El que sí que podem dir és que el número de moviments és el mateix en qualsevol camí òptim.
2. El camí és únic llevat de permutacions.
3. Com més estocàstic és l'entorn més es triga a arribar a una solució, però sempre serà un dels camins òptims possibles.

3. El canvi més radical que hem fet ha sigut canviar la implementació de la Q -table. En l'exercici 1 teníem un entorn controlat amb només un jugador i molt fàcil de modelar. Per això vam decidir implementar una matriu de dimensió $3 \times 4 \times 4$ on la primera dimensió són les files, les columnes i la tercera els diferents Q values de cada acció.

Per altra banda, en la segona part de la pràctica no era bona idea seguir aquesta idea ja que teníem una matriu 8×8 per modelar els escacs i la quantitat d'estats possibles és molt alta. Per això hem decidit utilitzar un diccionari on guardem les parelles clau valor de la següent forma: state: Q-value.

A més a més un altre canvi que hem hagut de fer ha sigut adaptar les recompenses segons la posició de les peces. Evidentment, també ens hem hagut de tornar a familiaritzar amb les funcions de la P_1 i de la P_2 .

4. Primer de tot fem una petita comparació entre l'algorisme determinista i l'estocàstic. Utilitzant els paràmetres comentats en preguntes anteriors podem notar una diferencia molt gran en els temps d'execució. Per una banda, l'algorisme determinista té els següents temps d'execució:

39.08, 26.80, 28.68, 44.05, 34.6

i obtenim una mitja de 34.64 segons per execució. Per l'altra banda l'algorisme estocàstic triga

124.91, 109.63, 100.55, 68.19, 93.42,

de mitja uns 99.34 segons, això és quasi 3 cops més. El motiu d'aquest resultat és que com els valors d'aprenentatge α, β són molt petits el programa necessita moltes més execucions per aprendre i, per tant, triga més temps. Val a dir que els moviments fets són sempre els mateixos. És curiós perquè el rei en comptes de moure's en línia recta es mou en diagonal evitant el rei negre. Creiem que es fa per minimitzar el perill de ser atacat. Si consultem la sortida de la P1 amb l'algorisme A^* , veiem:

1. La mitjana dels temps d'execució és de 0,02 segons, terriblement més baixa que en el cas del Q -learning. Creiem que això és degut al procés d'aprenentatge de l'algorisme.
2. En llenguatge més planer, podríem dir que l' A^* ja sap al que va des del principi, mentre que el Q -learning va més perdut que *un pulpo en un garaje*.
5. Fent diverses proves, hem vist que la nostra tria de paràmetres ha estat bona o, com a mínim, consistent. Els temps d'execució són similars, i el nombre d'iteracions fins que s'adquireix convergència també. En les proves que hem fet les diferències temporals estaven entre els 4-5 segons, bastant assumible. ■

CONCLUSIONS

L'objectiu d'aquesta pràctica, que era familiaritzar-nos amb Q -learning, un algorisme bàsic d'aprenentatge per reforç (RL), ha estat assolit satisfactòriament. Hem vist com els algorismes RL aprenen sobre la base de l'avaluació d'assaig i error. Hem vist i quantificat la convergència d'un algorisme, que és quan les recompenses obtingudes s'acosten als valors esperats. Hem pogut treballar sobre dos casos, un taulell simple i, per altra banda, els escacs. En el cas dels escacs, el problema consisteix a trobar una política que condueixi a un escac i mat. Ens hem vist obligats a determinar, per exemple, com definim els estats, com definim la política (quina usem, en el nostre cas ϵ -greedy), com definim la taula de Q -values... Hem pogut veure de primera mà com treballa el Q -learning; en particular, la seva adaptabilitat en entorns estocàstics, permetent l'aprenentatge d'estratègies en situacions amb incertesa i transicions probabilístiques.

En definitiva, aquesta pràctica ha sigut un gran primer pas en el món dels algorismes bàsics d'aprenentatge per reforç.

REFERÈNCIES

- [SB20] Richard S. SUTTON i Andrew G. BARTO. *Reinforcement learning: an introduction*. eng. Second edition. Adaptive computation and machine learning. Cambridge, Massachusetts: The MIT Press, 2020. ISBN: 9780262039246.

- [RNC22] Stuart J. (Stuart Jonathan) RUSSELL, Peter NORVIG i Ming-Wei CHANG. *Artificial intelligence : a modern approach*. eng. Fourth Edition, Global Edition. Pearsons series in artificial intelligence. Harlow: Pearson Education Limited, 2022. ISBN: 9781292401133.